

ZenFlow — Perché investire

- Riduce attriti tra pianificazione, stima ed esecuzione
- Funzionalità chiave integrate (meno plugin, meno switch)
- Pensato per team multiprogetto e contesti B2B multi-tenant

Data: 2 settembre 2025

Executive Summary

- Problema: tool generalisti richiedono plugin/integrazioni per coprire il ciclo agile end-to-end.
- Soluzione: ZenFlow integra pianificazione sprint, Planning Poker, metriche e sicurezza enterprise.
- Valore: meno contesto perso, meno costi di add-on, adozione più rapida.

Target: team di sviluppo (PM/Scrum Master/Dev) in PMI e aziende multiprogetto.

Perché non Jira (o non solo)

- Planning Poker: non nativo; in Jira serve marketplace → in ZenFlow è integrato.
- Coerenza del dato: velocità e punti propagati automaticamente su sprint e progetto.
- Semplicità: flusso SCRUM curato (story → sprint → stima → tracking) senza configurazioni complesse.
- Multi-tenant: isolamento per clienti/BU su un'unica installazione (Jira tipicamente multi-istanza).
- RBAC per-permesso: controllo fine senza dipendere da progetti/ruoli globali.

Nota: Jira offre molte funzionalità potenti; ZenFlow punta a velocità di adozione e focus sul ciclo agile.

Benefici & ROI attesi

- Meno context switching: stima, sprint e tracking nello stesso flusso.
- Time-to-value: setup rapido, processi opinionated ma estendibili.
- Riduzione costi add-on: Planning Poker e metriche incluse.
- Qualità metriche: calcolo automatico e coerente della velocità.

KPI suggeriti: tempo medio planning, durata stima, varianza punti stimati/visti.

Panoramica Tecnica

- Backend: Spring Boot + Thymeleaf (MVC)
- Sicurezza: Spring Security con RBAC per-permesso
- Multitenancy: routing dinamico del DataSource + filtri tenant
- Migrazioni: Flyway separato per master/tenant
- I18n: bundle `messages` IT/EN + switch lingua
- Email: JavaMail + Thymeleaf template (reset password)

Riferimenti codice: `config/*`, `config/multitenant/*`, `resources/messages/*`

Sicurezza & Compliance

- Login/Logout con pagina dedicata (`/login`)
- Reset password via email con token (24h)
- Invito utente con password temporanea e cambio password obbligatorio
- Session management (1 sessione per utente)
- Autorizzazioni a livello metodo (es. `@PreAuthorize` per CRUD)
- Audit revision (Hibernate Envers) con utente e IP

Riferimenti: `WebSecurityConfig` , `PasswordResetController` , `EmailService` ,
`UserAuditRevisionListener`

Internazionalizzazione (i18n)

- Locali supportate: Italiano e Inglese
- Switch runtime via endpoint `/change-lang`
- UI e messaggi tradotti (es. Admin, Dashboard, Errori)

Riferimenti: `messages_{it,en}.properties`, `LocaleController`

Dashboard Utente (overview)

- Progetti a cui partecipo
- User stories assegnate (Backlog / In corso / Completate)
- Sprint attivi dei miei progetti

Riferimenti: `DashboardController` , `facade/DashboardFacade` ,
`templates/dashboard.html`

Progetti (core)

- Elenco con paginazione, dettaglio, creazione/modifica/eliminazione
- Proprietario, membri team, stato e tipologia progetto
- Pagine: `projects/list` , `projects/detail` , `projects/form`

Riferimenti: `ProjectController` , `ProjectFacade` , `ProjectDTO` ,
`model.project.enums.ProjectStatus/Type`

User Stories (core)

- Elenco paginato per progetto e dettaglio
- CRUD con campi: stato, priorità, assegnatario, story points
- Tipi di stima: Story Points o PERT
- Associazione a sprint (da vista sprint)

Riferimenti: `UserStoryController`, `UserStoryFacade`, `UserStoryDTO`, `enums:`
`StoryStatus`, `Priority`, `EstimationType`

Sprint (SCRUM)

- Disponibili solo per progetti di tipo SCRUM
- Stati: PLANNED, ACTIVE, COMPLETED
- Azioni: avvia, completa, annulla sprint
- Gestione storie: aggiungi/rimuovi storie allo sprint

Riferimenti: `SprintController` , `SprintFacade` , pagine `projects/sprints/*`

Metriche & Velocità (differenziatore)

- Calcolo punti pianificati: somma story points delle storie nello sprint
- Calcolo punti completati: DONE-only
- Calcolo velocità: punti completati / durata sprint (in settimane)
- Aggiornamento velocità media progetto su sprints completati

Riferimenti: `SprintMetricsService`

Planning Poker (integrato)

- Sessioni per progetto e (opz.) specifica user story
- Partecipanti selezionati; facilitatore con controlli start/complete/cancel
- Voto con story points e motivazione; visibilità regolata da stato sessione
- Elenco globale e per progetto, regole autorizzative su ruolo/partecipazione

Riferimenti: `PlanningPokerController/Facade` , `EstimationVoteDTO` ,
`templates/planning-poker/*`

Amministrazione: Utenti

- Lista, dettaglio, modifica dati e ruoli
- Abilita/Disabilita utente
- Invita utente (email con password temporanea)

Riferimenti: `admin/UserController` , `UserInviteController` , `UserService` , pagine
`templates/admin/*`

Amministrazione: Ruoli & Permessi

- Ruoli con set di permessi granulari, raggruppati per categoria
- CRUD ruoli; visualizzazione permessi per ruolo
- Integrazione i18n per descrizioni permessi

Riferimenti: `admin/RoleController` , `PermissionService` , `messages_*` (chiavi `permission.*`)

Multitenancy: Master Dashboard

- Spazio amministrativo separato sotto `/master/*`
- Login dedicato (in-memory admin), navbar e stile master
- Gestione tenants: crea, visualizza, modifica, abilita/disabilita
- Inizializzazione DB tenant (Flyway) on-demand
- DataSource routing per-tenant via `AbstractRoutingDataSource`

Riferimenti: `master/*`, `MultitenantConfiguration`, `TenantFilter`,
`MasterTenantService`, pagine `templates/master/*`

Email & Template

- Reset password HTML con Thymeleaf (`password-reset-email.html`)
- Email invito semplice con messaggi localizzati
- Servizio email centralizzato con supporto template

Riferimenti: `EmailService` , `resources/templates/*`

UI/UX & Impostazioni

- Layout modulare con frammenti Thymeleaf (navbar, modali, pagination)
- Asset statici dedicati (`css/js/images`), inclusa animazione tema
- Impostazioni profilo: placeholder cambio tema (persistenza via localStorage)

Riferimenti: `templates/fragments/*`, `static/*`, `SettingsController`

Migrazioni Database

- Script Flyway per master e tenant separati
- Creazione ruoli/permessi iniziali e utente admin tenant
- Evolutive: aggiunta Planning Poker, refactor permessi

Riferimenti: `resources/db/migration/{master,tenant}/*.sql`

Roadmap (valore per il cliente)

- Resource management: visibilità carico/allocazione per persona; suggerimenti auto su risorse meno cariche.
- Dashboard avanzata: Gantt di progetto, burndown/burnup, velocity trend, report time tracking.
- LLM agent – Retrospective: genera draft di retro alla fine sprint, evidenzia pattern e azioni proposte.
- LLM agent – Story from spec: importa un capitolato tecnico e propone user stories con acceptance criteria.
- Integrazioni: Jira/GitHub/SSO; import/export sicuri.

Obiettivo: ridurre overhead e alzare qualità decisioni del team.

Link utili (per revisione rapida)

- Controller chiave: `controller/*`, `controller/admin/*`, `master/controller/*`
- Facade/Service: `facade/*`, `service/*`, `service/rbac/*`
- Template: `resources/templates/*` (views), `resources/messages/*` (i18n)
- Config: `config/*`, `config/multitenant/*`

Prossimi passi

- Pilot di 2–4 settimane su 1–2 team
- Setup tenant dedicato e migrazione minima (progetti/storie)
- Misura KPI prima/dopo (planning time, accuracy stime)

Contatti: team ZenFlow

Grazie!

Posso fornire demo guidata e materiali per stakeholders non tecnici.